

Project: Bank Customer Segmentation & Transaction Analysis

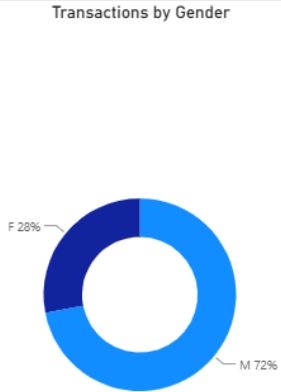
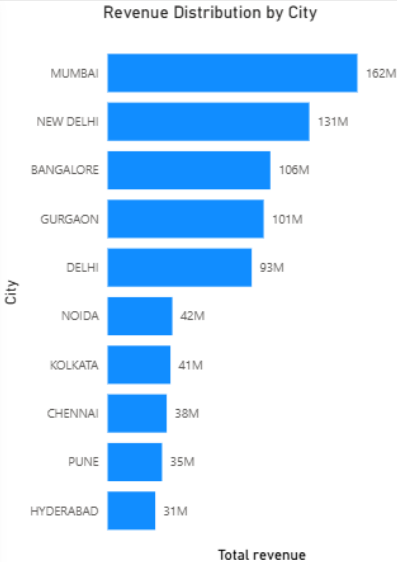
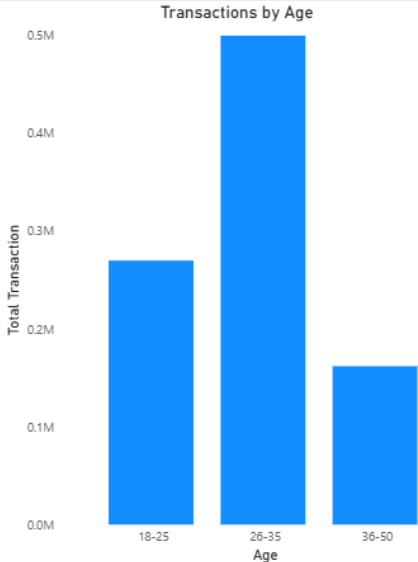
Bank Customer Segmentation Dashboard

780M

Total Revenue

1.45K

Average Transaction (INR)



Executive Summary & Insights:

- The Challenge:** Legacy queries caused server bottlenecks, and raw data contained severe errors (e.g., 57,000 invalid 1800 birth years).
Goal: Clean data for accurate segmentation.
- Key Insight 1 (The Gender Spending Gap):** While male customers dominate the overall transaction volume (72%), female customers (28%) spend significantly more per transaction. The average transaction value for women is 1,599.83 INR, compared to only 1,397.70 INR for men.
- Key Insight 2 (Top Performing Hub):** Mumbai is the undisputed leading market for the bank. It not only has the highest volume of transactions but also generates the highest total revenue, bringing in over 162 Million INR.
- Key Insight 3 (Data Quality Crisis):** The raw demographic data contained massive errors, such as 57,000 customers listed with a birth year of 1800. Filtering these out left us with a highly reliable dataset of 987,831 valid transactions for accurate analysis.

Business Recommendations:

- Recommendation 1:** Design and launch a premium credit card with higher fees and better rewards specifically targeted at female customers, as the data proves they have a higher spending power per transaction.
- Recommendation 2:** Focus marketing budgets, new ATM installations, and branch expansion campaigns primarily in Mumbai to maximize ROI, while tailoring ads for the highly active 26-35 age group.
- Business Impact:** By eliminating data bottlenecks and creating targeted profiles, the bank can confidently allocate marketing budgets to high-value segments (like female spenders) and high-revenue regions (Mumbai), ultimately improving customer retention.

Technical Execution & Data Pipeline:

1.Methodology & Data Process:

- Data Profiling & Discovery:** Scanned over 1 million raw transaction records in Google BigQuery to identify anomalies, successfully spotting critical demographic traps (e.g., missing genders, placeholder '1/1/1800' birthdates).
- Data Cleansing Logic:** Engineered robust SQL filtering logic using IS NOT NULL and exclusion operators to strip out dirty data, securing a reliable foundation of 987,831 rows.
- Query Optimization:** Replaced highly repetitive, server-draining queries by staging the clean data into an optimized Temporary Table (clean_bank_transactions).
- Scalability:** This modular approach (acting like a CTE) significantly reduced the query processing quota and made the logic easy to read, modify, and scale for the rest of the data team.

2. The SQL (Deconstructing Complexity & Advanced Segmentation):

- **Breaking Down Complexity:**
Used a CTE (WITH...AS) to divide complex business logic into clean, readable steps instead of writing one messy 100-line query.
- **Data Cleansing First:**
Filtered out bad data (like 57,000 users born in the year 1800) in the first step to ensure accurate calculations.
- **Actionable Segmentation:**
Dynamically grouped clean customer data into age brackets to guide marketing budgets effectively.

```
-- =====  
-- Task: Customer Age Segmentation  
-- Goal: Group active customers into age cohorts for targeted marketing.  
-- Logic:  
-- 1. PARSE_DATE: Converts string birthdates to DATE format.  
-- 2. DATE_DIFF: Calculates exact customer age at dataset baseline ('2016-12-31').  
-- 3. CASE WHEN: Segments users into structured age brackets.  
-- =====  
  
SELECT  
  CASE  
    WHEN DATE_DIFF(DATE '2016-12-31', PARSE_DATE('%d/%m/%y', CustomerDOB), YEAR) BETWEEN 18 AND 25 THEN  
      '18-25'  
    WHEN DATE_DIFF(DATE '2016-12-31', PARSE_DATE('%d/%m/%y', CustomerDOB), YEAR) BETWEEN 26 AND 35 THEN  
      '26-35'  
    WHEN DATE_DIFF(DATE '2016-12-31', PARSE_DATE('%d/%m/%y', CustomerDOB), YEAR) BETWEEN 36 AND 50 THEN  
      '36-50'  
    WHEN DATE_DIFF(DATE '2016-12-31', PARSE_DATE('%d/%m/%y', CustomerDOB), YEAR) > 50 THEN '50+'  
    ELSE 'Unknown'  
  END AS Age_Group,  
  COUNT(*) AS Total_Transactions  
FROM `data-analysis-projects-496119.bank_transactions.clean_bank_transactions`  
WHERE CustomerDOB IS NOT NULL  
GROUP BY Age_Group  
ORDER BY Total_Transactions DESC;
```